IPvX

Copyright © 2021 by Ron Maupin All Rights Reserved.

Table of contents

Welcome to IPvX 3
Getting Started with IPvX 4
Introduction 5
About IPvX 6
System Requirements 7
Getting help 8
License Information 9
The User Interface 10
Menus 12
File Menu 13
IP Menu 14
Hwlp Menu 15
Keypad 16
IPv4 Keypad 17
IPv6 Keypad 18
Value Fields and Labels 19
IPv4 Values and Labels 20
IPv6 Values and Labels 22
Status Bar 24
Object Definitions 25
IPv4 Object Definition 26
IPv6 Object Definition 27

Welcome to IPvX

IPvX

IPvX is a simple, easy-to-use, and intuitive IP calculator for both IP versions.

IPvX provides a clear and efficient user interface for IP calculations. IPvX can be used from the keyboard, or only the mouse, thanks to its built-in keypad that modifies itself for the IP you have chosen to use.

Getting Started with IPvX

New to IPvX?

- Read the Introduction section to learn more about IPvX.
- Read and follow the IANA documentation and IETF RFCs for the IPs.

Introduction

- About IPvX
- System Requirements
- Getting Help
- License Information

About IPvX

IPvX is a tool for IP, both IPv4 and IPv6, calculations that began as a test for my IP library where there is one object for IPv4 and another object for IPv6. The IPv4 object properties are read and written as strings in the common dotted-decimal format (without leading zeroes in the octets), and the IPv6 object properties are written in any of the three *RFC 4291, Section 2.2* *conventional* text representations for IPv6, but IPv6 object properties are always read in the *RFC 5952, Section 4* *canonical* text representation for IPv6 addresses. Each object only stores two values: Address and Mask (IPv4 32-bit, unsigned integers, and IPv6 128-bit, unsigned integers). All the other properties of an object are calculated when they are read, and the read-write properties calculate and modify the IP object Address and/or Mask values when they are written. IPvX instantiates one object for each IP.

As IPvX evolved, various features, such as information about the IP addresses (cast, scope, and description) in a status bar, and a keypad (allowing mouse-only use of IPvX) were added to the application. The keypad was inspired by the IPv6 Buddy.

The IP address status was informed by the *IANA IPv4 Special-Purpose Address Registry*, *IANA IPv4 Multicast Address Space Registry*, *IANA IPv6 Special-Purpose Address Registry*, *IANA IPv6 Multicast Address Space Registry*, and various RFCs.

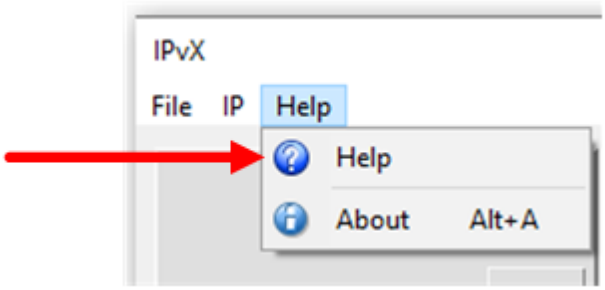
System Requirements

The IPvX recommended system configuration includes:

- Windows 10 or later
- 512 MB of RAM
- 5 MB of free disk space
- 1024x768 screen resolution or higher

Getting help

The IPvX help file is IPvX.chm. To launch it, either press the **<F1>** key for context sensitive help, or choose the **Help** menu item from the **Help** menu.



License Information

Author:

Ron Maupin

Copyright:

IPvX © 2010 - 2021 by Ron Maupin
IP.pas © 2010 - 2021 by Ron Maupin
Velthuis.BigIntegers.pas © 2015,2016,2017 Rudy Velthuis

Credits:

Thanks to Rudy Velthuis for the BigInteger library to simplify IPv6 128-bit math. IPv6 addresses can be handled as an array of two 64-bit or four 32-bit, unsigned integers, but it is simpler and easier to be able to use 128-bit, unsigned integers which is what an IPv6 address is.

License:

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met: *

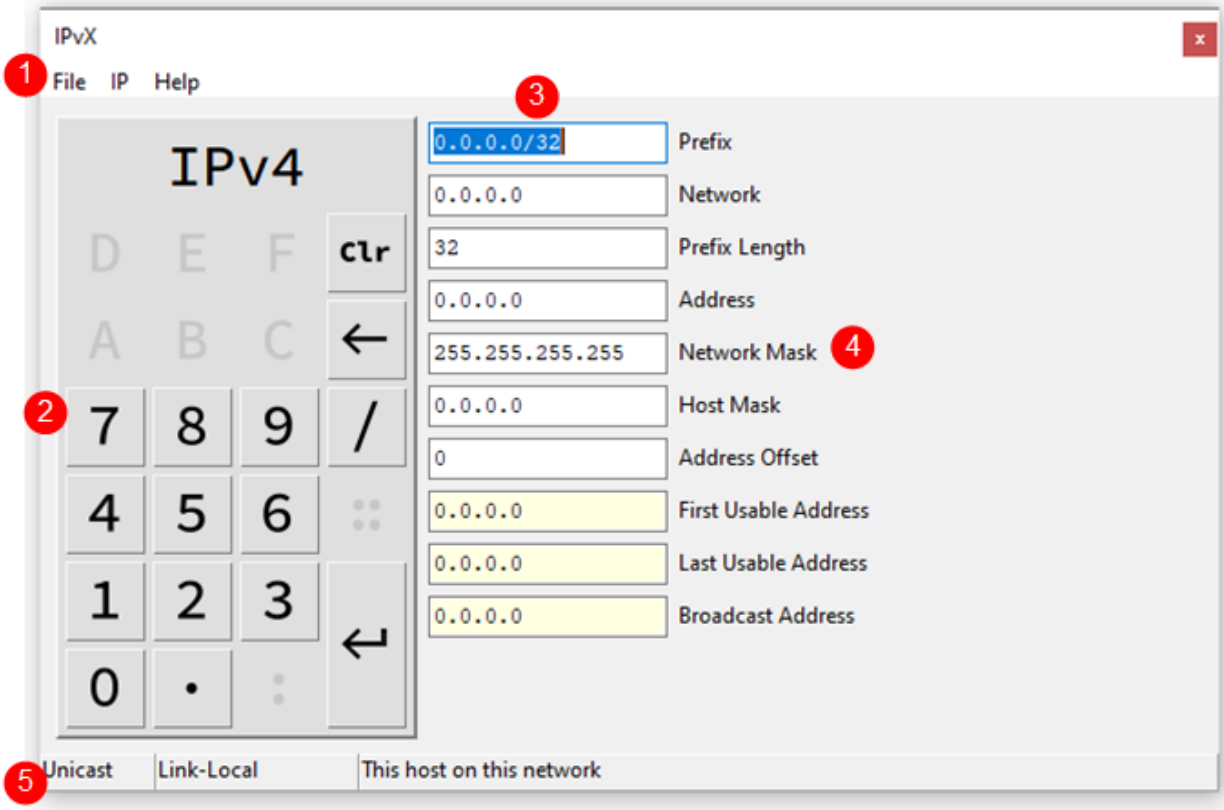
- Redistribution of source code must retain the above copyright notices, this list of conditions and the following disclaimer.
- Redistribution in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution. *

Disclaimer:

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

The User Interface

The IPvX user interface:



1. Menus

- Reset the current IP object.
- Change the current IP object.
- Launch the Help file.
- Launch the About form.
- Exit IPvX.

2. Keypad

- Facilitates mouse-only use of IPvX.
- Adapts its configuration for the current IP object.

3. Value Fields

- Display the property values of the current IP object.
- Read-write value fields allow changing of the properties for the current IP object.
- Adapt to the current IP object.

4. Labels

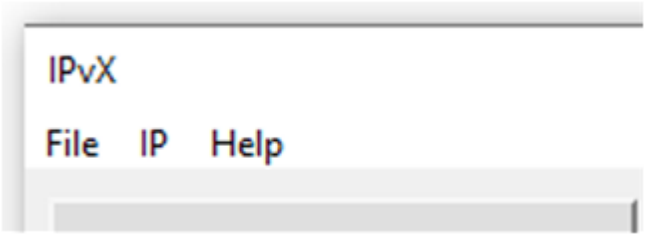
- Identify the corresponding value fields.
- Adapt to the current IP object.

5. Status Bar

- Displays the address cast.
- Displays the address scope.
- Displays the address description.
- Adapts to the current IP object.

Menus

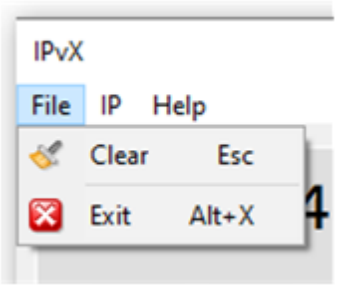
IPvX has three simple menus: **File**, **IP**, and **Help**.



- File Menu
- IP Menu
- Help Menu

File Menu

The **File** menu has two menu items: **Clear** and **Exit**.



Clear

Selecting the **Clear** menu item, or using the **<Esc>** key shortcut:

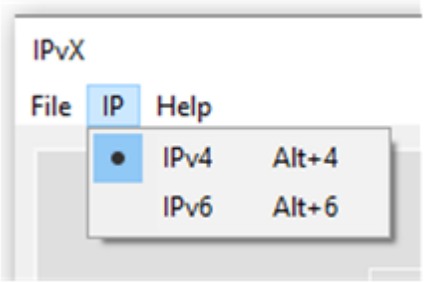
- Resets the currently selected IP object (sets the IP object's address to all-zeroes and the IP object's mask to all ones)
- Reads all the IPv4 object properties
- Updates the value fields with the IP object's property values
- Updates the status bar fields for the IP object's address

Exit

Selecting the **Exit** menu item, or using the **<Alt+X>** key shortcut, will close the IPvX application.

IP Menu

The **IP** menu has two menu items: **IPv4** and **IPv6**. The currently selected IP object is indicated by the dot image to the left of the menu item, and it is shown at the top of the keypad.



IPv4

Selecting the **IPv4** menu item, or using the **<Alt+4>** key shortcut:

- Selects the IPv4 object
- Reads all the IPv4 object properties
- Updates the value fields with the IPv4 object property values
- Updates the labels for the IPv4 object properties
- Updates the status bar fields for the IPv4 object address

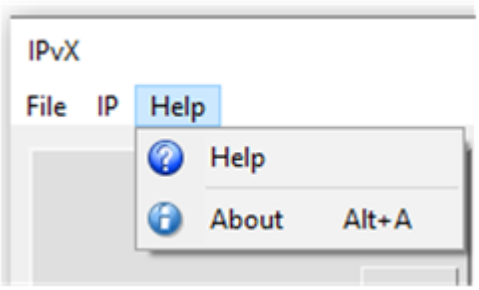
IPv6

Selecting the **IPv6** menu item, or using the **<Alt+6>** key shortcut:

- Selects the IPv6 object
- Reads all the IPv6 object properties
- Updates the value fields with the IPv6 object property values
- Updates the labels for the IPv6 object properties
- Updates the status bar fields for the IPv6 object address

Hwlp Menu

The **Help** menu has two menu items: **Help** and **About**.



Help

Selecting the **Help** menu item, or pressing the **<F1>** key for context sensitive help, launches the Help file.

About

Selecting the **Help** menu item, or using the **<Alt+A>** key shortcut, launches the About form that has information about the IPvX application.

Keypad

The keypad can be used to enter values into the value fields, allowing for mouse-only use of IPvX. The keypad adapts itself for the selected IP version, showing the IP version at the top of the keypad. Selecting IPv4 disables the keypad the IPv6-only keys. Selecting IPv6 enables all the keypad keys.

The keypad was inspired by the IPv6 Buddy.

- IPv4 Keypad
- IPv6 Keypad

IPv4 Keypad

The IPv4 keypad shows **IPv4** at the top, and it enables just the keys necessary to use IPvX for IPv4 addressing:

0 - 9	**Numeric Keys**	The numeric keys are used to enter numeric digits in the value fields. Clicking on a numeric key is the equivalent of pressing a numeric key on the physical keyboard.
.	**Dot Key**	The dot key is used to separate octets. Clicking on the dot key is the same as pressing the period or decimal key on the physical keyboard.
/	**Slash Key**	The slash key is used to enter a slash preceding a prefix length. Clicking on the slash key is the equivalent of pressing the slash key on the physical keyboard.
↵	**Enter Key**	The enter key is used to submit a value to the object property of the current IP object. Clicking on the enter key is the equivalent of pressing the enter key on the physical keyboard.
←	**Backspace**	The backspace key is used to erase the digit to the left of the cursor. Clicking on the backspace key is the equivalent of pressing the backspace key on the physical keyboard.
Clr	**Clear Key**	The clear key is used to reset the current IP object to the default values. Clicking the key is the same as selecting the Clear menu item or pressing the escape key on the physical keyboard.

IPv6 Keypad

The IPv6 keypad shows **IPv6** at the top, and it enables just the keys necessary to use IPvX for IPv6 addressing:

0 - 9	**Numeric Keys**	The numeric keys are used to enter numeric digits in the value fields. Clicking a numeric key is the same as of pressing a numeric key on the physical keyboard.
A - F	**Alphabetic Keys**	The alphabetic keys are used to enter alphabetic digits in the value fields. Clicking an alphabetic key is the same as of pressing an alphabetic key on the physical keyboard.
.	**Dot Key**	The dot key is used to separate octets for mixed-mode addresses. Clicking the key is the same as pressing the period or decimal key on the physical keyboard.
:	**Colon Key**	The colon key is used to separate the hexadecimal fields (16-bit words) in the value fields. Clicking the key is the same as of pressing the colon key on the physical keyboard.
/	**Slash Key**	The slash key is used to enter a slash preceding a prefix length. Clicking the key is the same as of pressing the slash key on the physical keyboard.
::	**Double-Colon Key**	The double-colon key is used to enter two colons for the compressed format. Clicking the double-colon key is the same as pressing the colon key twice on the physical keyboard.
↵	**Enter Key**	The enter key is used to submit a value to the object property of the current IP object. Clicking on the enter key is the same as of pressing the enter key on the physical keyboard.
←	**Backspace**	The backspace key is used to erase the digit to the left of the cursor. Clicking on the backspace key is the same as of pressing the backspace key on the physical keyboard.
Clr	**Clear Key**	The clear key is used to reset the current IP object to the default values. Clicking the key is the same as selecting the Clear menu item or pressing the escape key on the physical keyboard.

Value Fields and Labels

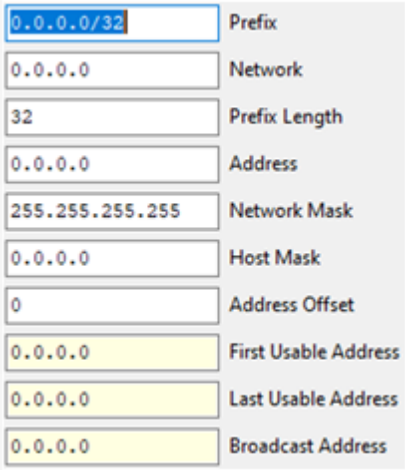
The value fields and labels will vary, depending on the IP object selected. Each value field has a corresponding label to describe what the value field contains. Changing the IP object will change the visible value fields and their labels.

The value fields display the properties of the currently selected IP object. The values in each value field can be selected and copied into the Windows clipboard to be pasted into a different application. Most value fields are read-write fields that can be changed by the user to update the address and/or mask of the currently selected IP object, and the user can enter values into the read-write value fields with the keyboard, IPvX keypad, or paste values from the Windows clipboard.

- IPv4 Values and Labels
- IPv6 Values and Labels

IPv4 Values and Labels

The IPv4 object has 10 properties, the first seven of which are read-write properties, and the last three are read-only properties. The IPv4 object only stores an address and mask, and all the other properties are calculated when read, and the read-write properties modify the IPv4 object address and/or mask.



Prefix

- **Read** - The value string is created by masking the IPv4 object address with the IPv4 object mask to get the network address, which is converted to string in the common IPv4 dotted-decimal notation. The prefix length is created by counting the number of bits in the IPv4 object mask and converting the number of bits to a string. A slash and the prefix length string are appended to the network address string.
- **Write** - The string value is tested by a regular expression to determine if it is a valid IPv4 dotted-decimal value, including a slash and valid prefix length. The network address part of the string is converted to a 32-bit, unsigned integer. The prefix length part of the string is converted to an integer, from which a mask is calculated by setting the integer number of bits in a 32-bit, unsigned integer. The calculated mask replaces the IPv4 object mask. The calculated network address is masked with the IPv4 object mask to arrive at the network address (the network address is truncated by the IPv4 object mask). The masked network address is placed in the network portion of the IPv4 object address.

Network

- **Read** - The value string is created by masking the IPv4 object address with the IPv4 object mask to get the network address, which is converted to string in the common IPv4 dotted-decimal notation.
- **Write** - The string value is tested by a regular expression to determine if it is a valid IPv4 dotted-decimal value. The network address string is converted to a 32-bit, unsigned integer. The calculated network address is masked with the IPv4 object mask to arrive at the network address (the network address is truncated by the IPv4 object mask). The masked network address is placed in the network portion of the IPv4 object address. ***Optional***: A prefix length may be included in the value string, and the network property is handled the same as the prefix property.

Prefix Length

- **Read** - The value string is is created by counting the number of bits in the IPv4 object mask and converting the result to an integer string.
- **Write** - The string value is converted to an integer and tested to see that it falls in the valid IPv4 length range (**`0`** to **`32`**). The calculated value is used to calculate a network mask by setting the integer number of bits in a 32-bit, unsigned integer. The calculated mask replaces the mask in the IPv4 object.

Address

- **Read** - The value string is created by converting the IPv4 object address to a string in the common IPv4 dotted-decimal notation.

- **Write** - The string value is tested by a regular expression to determine if it is a valid IPv4 dotted-decimal value. The string is converted to a 32-bit, unsigned integer. The calculated address replaces the address in the IPv4 object. ***Optional***: A prefix length may be included in the value string written to the network property, and the prefix length is handled the same as the prefix length to replace the IPv4 object mask.

Network Mask

- **Read** - The value string is created by converting the IPv4 object mask to a string in the common IPv4 dotted-decimal notation.
- **Write** - The string value is tested by a regular expression to determine if it is a valid IPv4 dotted-decimal value. The string is converted to a 32-bit, unsigned integer, and it is tested to check that it consists of consecutive one bits. The 32-bit, unsigned integer replaces the mask in the IPv4 object.

Host Mask

- **Read** - The value string is created by converting the inverse of the IPv4 object mask to a string in the common IPv4 dotted-decimal notation.
- **Write** - The string value is tested by a regular expression to determine if it is a valid IPv4 dotted-decimal value. The string is converted to a 32-bit, unsigned integer, and the inverse is tested to check that it consists of consecutive one bits. The inverse of the 32-bit, unsigned integer replaces the mask in the IPv4 object.

Address Offset

- **Read** - The value string is created by masking the IPv4 object address with the inverse of the IPv4 object mask, which is converted to an integer string.
- **Write** - The string value is converted to a 32-bit, unsigned integer. The 32-bit, unsigned integer is tested to verify that it is less than or equal to the inverse of the IPv4 object mask. The host portion of the IPv4 address is replaced with the host portion of the 32-bit, unsigned integer.

First Usable Address

- **Read** - The value string is created by masking the IPv4 object address with the IPv4 object mask to get the network address, to which one is added if the prefix length is less than **`31`**, which is converted to string in the common IPv4 dotted-decimal notation..

Last Usable Address

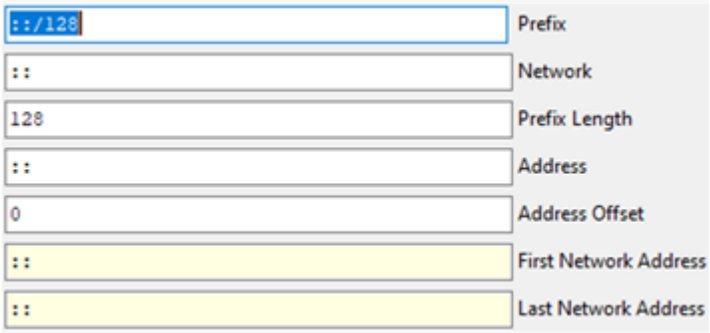
- **Read** - The value string is created by masking the IPv4 object address with the IPv4 object mask to get the network address, to which the inverse of the IPv4 object mask is added, and one is subtracted if the prefix length is less than **`31`**, which is converted to string in the common IPv4 dotted-decimal notation..

Broadcast Address

- **Read** - The value string is created by masking the object address with the IPv4 object mask to get the network address, to which the inverse of the IPv4 object mask is added, which is converted to string in the common IPv4 dotted-decimal notation.

IPv6 Values and Labels

The IPv6 object has seven properties, the first five of which are read-write properties, and the last two are read-only properties. The IPv6 object only stores an address and mask, and all the other properties are calculated when read, and the read-write properties modify the IPv6 object address and/or mask.



Prefix

- **Read** - The value string is created by masking the IPv6 object address with the IPv6 object mask to get the network address, which is converted to string in the RFC 5952 IPv6 canonical text representation. The prefix length is created by counting the number of bits in the IPv6 object mask and converting the number of bits to a string. A slash and the prefix length string are appended to the network address string.
- **Write** - The string value is tested by a regular expression to determine if it is a valid RFC 4291 IPv6 conventional text representation, including a slash and valid prefix length. The network address part of the string is converted to a 128-bit, unsigned integer. The prefix length part of the string is converted to an integer, from which a mask is calculated by setting the integer number of bits in a 128-bit, unsigned integer. The calculated mask replaces the IPv6 object mask. The calculated network address is masked with the IPv6 object mask to arrive at the network address (the network address is truncated by the IPv6 object mask). The masked network address is placed in the network portion of the IPv6 object address.

Network

- **Read** - The value string is created by masking the IPv6 object address with the IPv6 object mask to get the network address, which is converted to string in the RFC 5952 IPv6 canonical text representation.
- **Write** - The string value is tested by a regular expression to determine if it is a valid RFC 4291 IPv6 conventional text representation. The network address string is converted to a 128-bit, unsigned integer. The calculated network address is masked with the IPv6 object mask to arrive at the network address (the network address is truncated by the IPv6 object mask). The masked network address is placed in the network portion of the IPv6 object address. ***Optional***: A prefix length may be included in the string, and the network property is handled the same as the prefix property.

Prefix Length

- **Read** - The value string is is created by counting the number of bits in the IPv6 object mask and converting the result to a string.
- **Write** - The string value is converted to an integer and tested to see that it falls in the valid IPv6 length range (**`0`** to **`128`**). The calculated value is used to calculate a network mask by setting the integer number of bits in a 128-bit, unsigned integer. The calculated mask replaces the mask in the IPv6 object.

Address

- **Read** - The value string is created by converting the IPv6 object address to a string in the RFC 5952 IPv6 canonical text representation.
- **Write** - The string value is tested by a regular expression to determine if it is a valid RFC 4291 IPv6 conventional text representation. The string is converted to a 128-bit, unsigned integer. The calculated address replaces the address in the IPv6 object. ***Optional***: A prefix length may be included in the string written to the network property, and the prefix length is handled the same as the prefix length to replace

the IPv4 object mask.

Address Offset

- **Read** - The value string is created by masking the IPv6 object address with the inverse of the IPv6 object mask. The resulting 128-bit, unsigned integer is converted to a string.
- **Write** - The string value is converted to a 128-bit, unsigned integer. The 128-bit, unsigned integer is tested to verify that it is less than or equal to the inverse of the IPv6 object mask. The host portion of the IPv6 address is replaced with the host portion of the 128-bit, unsigned integer.

First Network Address

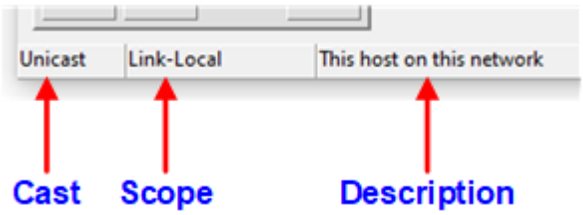
- **Read** - The value string is created by masking the IPv6 object address with the IPv6 object mask to get the network address, which is converted to string in the RFC 5952 IPv6 canonical text representation.

Last Network Address

- **Read** - The value string is created by masking the IPv6 object address with the IPv6 object mask to get the network address, to which the inverse of the IPv6 object mask is added, which is converted to string in the RFC 5952 IPv6 canonical text representation.

Status Bar

The Status Bar has three fields that describe the IP address of the currently selected IP object:



Cast

The cast field describes the cast of the address in the currently selected IP object:

- Unicast addresses can be used as both source and destination addresses, and are destined for a single host.
- Multicast addresses are group addresses that can only be used as destination addresses, and are destined only for hosts subscribed to the multicast group
- Broadcast addresses are group addresses that can only be used as destination addresses, and are destined for all hosts on a network (not used in IPv6)
- Reserved addresses cannot be used

Scope

There are various address scopes, depending on the IP version and cast, that were gleaned from the various IANA documents and IETF RFCs.

Description

The address descriptions are more-or-less official descriptions that were gleaned from the various IANA documents and IETF RFCs.

Object Definitions

There are two IP objects defined in the IP library used for IPvX:

- IPv4 Object Definition
- IPv6 Object Definition

IPv4 Object Definition

```
TIPv4 = class(TObject)                                                  // The IPv4 object;
  private
    type TAddr = UInt32;                                                // IPv4 addresses a
    var FAddr: TAddr;                                                   // The IPv4 Address
    var FMask: TAddr;                                                   // The IPv4 Mask
    function GetPrefix(): string;                                       // Calculates the P
    procedure SetPrefix(const APrefix: string);                         // Sets the Address
    function GetNetwork(): string;                                      // Calculates the N
    procedure SetNetwork(const ANetwork: string);                       // Sets the Address
    function GetNetLength(): string;                                    // Calculates the P
    procedure SetNetLength(const ANetLength: string);                   // Sets the Mask fr
    function GetAddress(): string;                                      // Returns the Addr
    procedure SetAddress(const AAddress: string);                       // Sets the Address
    function GetNetMask(): string;                                      // Returns the Mask
    procedure SetNetMask(const ANetMask: string);                       // Sets the Mask fr
    function GetHostMask(): string;                                     // Calculates the H
    procedure SetHostMask(const AHostMask: string);                     // Sets the Mask fr
    function GetOffset(): string;                                       // Calculates the A
    procedure SetOffset(const AOffset: string);                         // Sets the Address
    function GetFirst(): string;                                        // Calculates the F
    function GetLast(): string;                                         // Calculates the L
    function GetBroadcast(): string;                                    // Calculates the B
  public
    constructor Create();
    procedure Clear();                                                  // Clears the Addre
    procedure Assign(const Source: TIPv4);                              // Assigns the Addr
    function InRange(const APrefix: string): Boolean;                   // Test if the Addr
    destructor Destroy(); override;
    property Prefix:    string  read GetPrefix    write SetPrefix;      // The R/W Prefix p
    property Network:   string  read GetNetwork   write SetNetwork;     // The R/W Network
    property NetLength: string  read GetNetLength write SetNetLength;   // The R/W Prefix l
    property Address:   string  read GetAddress   write SetAddress;     // The R/W Address
    property NetMask:   string  read GetNetMask   write SetNetMask;     // The R/W Network
    property HostMask:  string  read GetHostMask  write SetHostMask;    // The R/W Host Mas
    property Offset:    string  read GetOffset    write SetOffset;      // The R/W Address
    property First:     string  read GetFirst;                          // The RO First Usa
    property Last:      string  read GetLast;                           // The RO Last Usab
    property Broadcast: string  read GetBroadcast;                      // The RO Broadcast
end;
```

IPv6 Object Definition

```
TIPv6 = class(TObject)                                          // The IPv6 object;
  private
    type TAddr = BigInteger;                                    // IPv6 addresses ar
    var FAddr: TAddr;                                           // The IPv6 Address
    var FMask: TAddr;                                           // The IPv6 Mask
    function GetPrefix(): string;                               // Calculates the Pr
    procedure SetPrefix(const APrefix: string);                 // Sets the Address
    function GetNetwork(): string;                              // Calculates the Ne
    procedure SetNetwork(const ANetwork: string);               // Sets the Address
    function GetNetLength(): string;                            // Calculates the Pr
    procedure SetNetLength(const ANetLength: string);           // Sets the Mask fro
    function GetAddress(): string;                              // Returns the Addre
    procedure SetAddress(const AAddress: string);               // Sets the Address
    function GetOffset(): string;                               // Calculates the Ad
    procedure SetOffset(const AOffset: string);                 // Sets the Address
    function GetFirst(): string;                                // Calculates the Fi
    function GetLast(): string;                                 // Calculates the La
  public
    constructor Create();
    procedure Clear();                                          // Clears the Addres
    procedure Assign(const Source: TIPv6);                      // Assigns the Addre
    function InRange(const APrefix: string): Boolean;           // Test if the Addre
    function Expand(const AAddress: string): string;            // Fully expands an
    function Compress(const AAddress: string): string;          // Compresses an IPv
    destructor Destroy(); override;
    property Prefix:    string  read GetPrefix    write SetPrefix;   // The R/W Prefix pr
    property Network:   string  read GetNetwork   write SetNetwork;  // The R/W Network p
    property NetLength: string  read GetNetLength write
SetNetLength;                                                   // The R/W Prefix Le
    property Address:   string  read GetAddress   write SetAddress;  // The R/W Address p
    property Offset:    string  read GetOffset    write SetOffset;   // The R/W Address O
    property First:     string  read GetFirst;                      // The RO First Netw
    property Last:      string  read GetLast;                       // The RO Last Netwo
end;
```